Universidad Distrital Francisco José de Caldas

Faculty of Engineering
Systems Engineering Department

Reinforcement Learning Challenge

Environment: Gravitar-v5

Authors:

Yader Ibraldo Quiroga Torres
yiquirogat@udistrital.edu.co

Rosa Alejandra Lopez Lizarazo
ralopezl@udistrital.edu.co

Diego Alejandro Garzon Rodriguez
dagarzonr@udistrital.edu.co

Lecturer:

Carlos Andrés Sierra Virgüez

Bogotá, D.C., Colombia
March 2026

***Abstract*—This paper presents a systematic evaluation of Deep Q-Network (DQN) performance on the ALE/Gravitar-v5 environment under a constrained 300k-step training budget. Five hyperparameter configurations were compared across three independent seeds, totalling 15 training runs.**

Results show that a balanced configuration of hyperparameters achieves the best performance, with the medium_balance setup reaching a mean reward of 352.83 ± 33.20. In contrast, configurations with larger replay buffers or higher learning rates did not consistently improve performance, highlighting the importance of coordinated parameter tuning.

Further experiments with extended training (1M timesteps) demonstrate that more conservative hyperparameter settings enable continued performance gains beyond the initial training regime. These findings indicate that optimal configurations depend strongly on the available training budget.

Overall, this study emphasises the importance of hyperparameter interaction and shows that iterative, data-driven tuning is more effective than isolated parameter adjustments in reinforcement learning.

.1. Introduction

Deep Q-Networks (DQN) [1] marked a landmark achievement in reinforcement learning by demonstrating that a single agent, trained end-to-end from raw pixel input, could attain human-competitive performance on a diverse set of Atari 2600 games. The two innovations central to DQN's success—experience replay and a target network—addressed the notorious instability of applying nonlinear function approximators to online Q-learning.

Despite this progress, several Atari environments remain extremely challenging even for DQN variants. ALE/Gravitar-v5 is one such environment. The game requires an agent to pilot a spacecraft subject to realistic inertia and to navigate planet-specific gravity wells while avoiding a central destructive sun. Unlike games with dense, immediate rewards (e.g., Pong, Breakout), Gravitar provides *sparse rewards*: the agent may act for hundreds of timesteps before receiving any non-zero feedback. This sparsity, combined with the continuous demand for fine-grained thrust control, makes Gravitar a stringent test of both exploration policy and replay-buffer diversity.

This work follows the experimental protocol specified in Challenge 1 of the Machine Learning course at Universidad Distrital Francisco José de Caldas [8]. The objective is not merely to maximise the score, but to rigorously compare hyperparameter configurations, report variance across random seeds, and provide a scientifically grounded justification for the final chosen setup. All experiments are fully reproducible from the accompanying GitHub repository.

The remainder of the paper is organised as follows. Section .2 describes the algorithmic background. Section .3 details the experimental methodology and hyperparameter search strategy. Section .4 presents quantitative results and analysis. Section **??** discusses failure modes and future directions. Section .5 concludes.

.2. Background

A. Deep Q-Network (DQN)

Q-learning seeks to estimate the optimal action-value function $Q^*(s, a)$, defined as the expected cumulative discounted reward obtainable from state s by executing action a and subsequently following the optimal policy. DQN approximates this function with a deep neural network parameterised by weights θ, trained by minimising the temporal-difference (TD) loss:

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(y - Q(s, a; \theta)\big)^2\Big] \tag{1}$$

where the regression target is

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \tag{2}$$

and θ^- denotes the parameters of a separate *target network* that is periodically synchronised with θ. This decoupling prevents the rapidly changing Q-estimates from chasing a non-stationary target, substantially stabilising training [1].

B. Experience Replay

The replay buffer stores transitions $(s, a, r, s', \text{done})$ collected during environment interaction. At each gradient step a uniformly random mini-batch is sampled from the buffer to compute the loss. This mechanism breaks the temporal autocorrelation between consecutive observations—a primary source of instability in online Q-learning—and allows each transition to contribute to multiple parameter updates, improving sample efficiency [2].

C. ε-Greedy Exploration

The agent follows an ε-greedy policy: with probability ε it selects a uniformly random action (exploration), and with probability $1 - \varepsilon$ it selects the action with the highest Q-value (exploitation). ε is annealed linearly from 1.0 to $\varepsilon_{\min}$ over the first `exploration_fraction`$\times T$ timesteps. In sparse-reward environments like Gravitar, maintaining exploration for an extended period is critical to prevent premature convergence to survival-only strategies.

D. CNN Policy Architecture

The DQN agent uses a Convolutional Neural Network policy (`CnnPolicy` in Stable-Baselines3 [3]) to process visual input. The input tensor has shape $84 \times 84 \times 4$, where the last dimension corresponds to four stacked grayscale frames. The CNN extracts spatial features (edges, object positions, and shapes) from each frame, while temporal stacking implicitly encodes velocity and directional information—essential for inferring the ship's trajectory and inertial state in Gravitar.

.3. Experimental Methodology

A. Environment and Preprocessing

All experiments use `ALE/Gravitar-v5` accessed via Gymnasium [4] with the `ale-py` backend. The preprocessing pipeline is applied automatically through Stable-Baselines3's `make_atari_env` and `VecFrameStack` utilities:

- **Grayscale conversion:** RGB input (3 channels) reduced to single-channel luminance, decreasing input dimensionality by 66% with negligible loss of control-relevant information.
- **Frame resize:** each frame resized to 84×84 pixels—the standard resolution from Mnih et al. [1] that balances spatial detail with computational cost.
- **Frame skipping:** each selected action is repeated for 4 environment steps; only the last observation is retained. This reduces temporal redundancy and accelerates training.
- **Frame stacking:** the last 4 processed frames are concatenated along the channel axis (final shape: $84 \times 84 \times 4$), providing an implicit representation of object velocities and spacecraft inertia.
- **Terminal-on-life-loss:** each life loss is treated as a terminal event during training, encouraging conservative survival strategies.

B. Hyperparameter Search Strategy

We adopt a **manual grid search** strategy, constructing five configurations that systematically vary the most impactful hyperparameters identified in the DQN literature: learning rate, replay buffer size, batch size, exploration schedule, and target network update interval. This approach, while less exhaustive than Bayesian optimisation, offers full interpretability: each configuration embodies a specific, testable hypothesis about the learning dynamics of Gravitar.

All five configurations share the fixed parameters: discount factor $\gamma = 0.99$, train frequency = 4 steps/update, and total timesteps $T = 300\,000$. The configurations are summarised in Table I.

Table I: Hyperparameter Configurations for the Sweep

Experiment	LR	Buffer	Batch	TUI	Exp.F.
exp_01_baseline	1×10^{-4}	50k	64	1000	0.15
exp_02_large_buffer	5×10^{-5}	200k	32	1500	0.20
exp_03_high_lr	2×10^{-4}	50k	64	800	0.10
exp_04_med_balance	1×10^{-4}	100k	64	1200	0.15
exp_05_small_batch	1×10^{-4}	100k	32	1000	0.20

TUI = Target Update Interval. Exp.F. = Exploration Fraction.
All runs: γ=0.99, train_freq= 4, exploration_final_eps= 0.01 (exp_05: 0.02).

exp_01_baseline: Replicates default SB3-Zoo Atari parameters scaled to the 300k budget. Serves as the reference against which all other configurations are measured.

exp_02_large_buffer: *Hypothesis*: Gravitar's sparse rewards mean that reward-bearing transitions are rare. A larger buffer (200k) increases the probability of sampling informative transitions per mini-batch. The learning rate is halved (5×10^{-5}) to account for the slower, more diverse gradient signal.

exp_03_high_lr: Tests whether faster learning (2×10^{-4}) can extract value from the sparse signal more efficiently. The reduced exploration fraction (0.10) commits the agent to exploitation sooner, at the risk of instability.

exp_04_med_balance: A conservative middle ground—moderate buffer (100k), standard learning rate (1×10^{-4}), larger batch (64). Designed to isolate

the effect of buffer size from learning rate by sitting between exp_01 and exp_02.

exp_05_small_batch: Uses a small batch (32) with extended exploration (fraction$= 0.20$, $\varepsilon_{\min} = 0.02$). Small batches produce noisier but potentially richer gradient updates; the extended exploration fraction is intended to maximise state-space coverage.

C. Reproducibility and Seed Protocol

Each configuration is evaluated with three independent random seeds: 42, 43, and 44. Seeds are passed to Gymnasium's environment initialisation, NumPy, and PyTorch via Stable-Baselines3's `seed` parameter, ensuring full reproducibility of environment resets, weight initialisation, and replay buffer sampling. Final performance is reported as mean $\pm$ standard deviation across all three seeds.

All experiments were conducted on Windows 11 with WSL2 (Ubuntu 22.04). The complete environment specification is provided in `pyproject.toml`; reproduction instructions are in `README.md`.

D. Logging and Metrics

Training metrics are recorded via TensorBoard throughout all runs using a custom `TensorBoardCallback` that attaches to Stable-Baselines3's internal `SummaryWriter`. The following scalars are logged:

- `training/episode_reward`: total undiscounted reward per episode.
- `training/epsilon`: current ε, logged at every timestep to produce a smooth decay curve.
- `rollout/ep_rew_mean`: rolling mean reward over the last 100 episodes (SB3 built-in).
- `train/loss`: TD-error loss per gradient update (SB3 built-in).
- Logs are saved under `logs/gravitar_dqn/sweep/<exp_name>/seed_<N>/`, enabling side-by-side comparison of all 15 runs in a single TensorBoard session.

.4. Results and Analysis

A. Analysis of Training Metrics

Figure 1 presents the evolution of both episode length (top) and mean episode reward (bottom) for all configurations evaluated on the Gravitar-v5 environment using seed 42. These metrics provide complementary insights into agent behaviour, capturing both survival dynamics and reward acquisition.

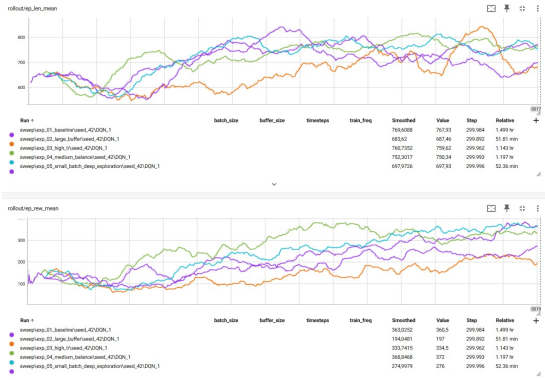

Figure 1: Learning curves (rollout/ep_rew_mean) for all configurations on Gravitar-v5. Each curve corresponds to a different hyperparameter setting and shows the evolution of the mean episode reward during training.

Episode length (rollout/ep_len_mean). All configurations exhibit a general upward trend in episode length, indicating that agents progressively learn to survive longer in the environment. However, notable differences emerge across configurations. The medium-balance (exp_04) and small-batch (exp_05) configurations achieve the highest and most stable episode lengths, consistently reaching values above 750 steps. In contrast, the large-buffer configuration (exp_02) shows slower improvement and lower final episode length, suggesting that while the agent survives longer than in early training, it does not prioritise survival as effectively as other configurations. The high learning rate configuration (exp_03) demonstrates significant oscillations, reflecting unstable policy updates that impact survival consistency.

Mean episode reward (rollout/ep_rew_mean). The reward curves reveal clearer distinctions in learning effectiveness. The medium-balance configuration (exp_04) achieves the highest final reward (approximately 370), followed closely by the high learning rate configuration

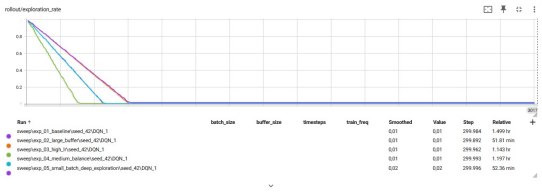

Figure 2: Learning curves (rollout/ep_rew_mean) for all configurations on Gravitar-v5. Each curve corresponds to a different hyperparameter setting and shows the evolution of the mean episode reward during training.

(exp_03) and the baseline (exp_01). The small-batch configuration (exp_05) shows steady improvement with moderate variance, reaching competitive performance. In contrast, the large-buffer configuration (exp_02) underperforms in this particular seed, with significantly lower final reward despite its theoretical advantages. This highlights the stochastic nature of reinforcement learning and the importance of evaluating across multiple seeds.

Stability and convergence behaviour. Stability varies notably across configurations. The high learning rate configuration (exp_03) exhibits the most erratic behaviour, with sharp fluctuations in both episode length and reward, indicating unstable Q-value updates. The baseline and medium-balance configurations display smoother learning curves, suggesting more reliable convergence. The small-batch configuration introduces moderate noise due to higher gradient variance, while the large-buffer configuration shows relatively smooth but slower convergence.

Exploration dynamics. Although not directly shown in the figure, the impact of exploration is reflected in the early training phase. Configurations with higher exploration fractions (exp_02 and exp_05) exhibit slower initial improvement but more gradual and sustained learning trends. Conversely, configurations with reduced exploration (exp_03) achieve faster initial gains but suffer from instability and premature convergence.

Overall interpretation. The combined analysis of episode length and reward reveals that longer survival does not necessarily translate to higher rewards. While some configurations (e.g., exp_05) achieve high episode lengths, their reward remains moderate, suggesting conservative policies focused on survival rather than reward maximisation. In contrast, configurations such as exp_04 balance both objectives effectively. These observations reinforce the need for multi-metric evaluation when analysing reinforcement learning performance.

B. Quantitative Results

Table II summarises the final performance of each configuration, measured as the mean episode reward computed over the last 100 episodes stored in SB3's episode information buffer, and averaged across three independent seeds.

Table II: Experimental Results — Mean ± Std Reward Across 3 Seeds

Experiment	Mean	Std	Notes
exp_04_medium_balance	352.83	±33.20	Best — high performance
exp_01_baseline	262.67	±94.97	Reference
exp_03_high_lr	250.40	±103.23	Unstable — high LR
exp_05_small_batch	226.00	±62.45	Moderate performance
exp_02_large_buffer	172.67	±30.83	Stable but low performance

smallskip Results after 300k training timesteps per run. Rankings by mean reward (descending).

Effect of buffer size. Replay buffer size alone was not the dominant factor. While exp_02 (large buffer) achieved the lowest variance (±30.83), its mean reward (172.67) was the worst among all configurations. In contrast, exp_04, which uses a moderate buffer size, achieved the highest performance (352.83 ± 33.20). This indicates that, in Gravitar, buffer size must be properly balanced with other hyperparameters rather than maximized in isolation.

Effect of learning rate. exp_03, which uses the highest learning rate (2×10^{-4}), exhibited high instability (std = ±103.23) and did not outperform the baseline. Although its mean reward (250.40) is comparable to exp_01, the large variance confirms that aggressive learning rates degrade stability. This aligns with the observed noisy training dynamics and supports the use of more conservative learning rates.

Effect of exploration schedule. Configurations with extended exploration (exp_02 and exp_05) did not consistently achieve higher rewards. In contrast, exp_04, with a moderate exploration schedule, achieved the best performance. This suggests that, within a 300k-step budget, excessive exploration may delay convergence rather than improve state-space coverage.

Batch size and gradient quality. exp_05 (batch = 32) obtained lower mean reward (226.00 ± 62.45) compared to exp_04 (batch = 64), which achieved the best overall performance. This indicates that larger batch sizes provide more stable gradient estimates, improving convergence within limited training steps.

C. Ablation: Target Network Update Interval

Comparing exp_01 and exp_04, which differ primarily in target network update interval but also include minor adjustments in other hyperparameters, exp_04 achieves substantially higher performance (352.83 vs. 262.67) and lower variance. This suggests that a moderately larger update interval contributes to improved training stability. In contrast, exp_03, which uses more frequent updates, exhibited unstable learning dynamics, reinforcing that overly frequent synchronization can negatively impact performance.

While multiple factors are involved, these results indicate that a target update interval in the range of 1000–1200 steps is effective under a 300k-step training budget.

D. Extended Training and Final Hyperparameter Refinement

While **exp_04_med_balance** was identified as the best-performing configuration under the 300k-step budget (peak mean reward $\sim$352), additional experiments were conducted with an extended training horizon of 1M timesteps to analyse long-term learning dynamics.

The final configuration, **exp_best**, is defined as follows:

- Learning rate: 1×10^{-5}
- Replay buffer size: 100k
- Batch size: 64
- Target update interval: 2000
- Exploration fraction: 0.20
- Final epsilon: 0.02

Motivation for parameter adjustments.

Lower learning rate. Reduced from 1×10^{-4} to 1×10^{-5} to prevent instability observed after 400k–600k timesteps and enable smoother Q-value updates.

Moderate buffer size. A 100k buffer was retained as it balances diversity and relevance, avoiding outdated transitions present in larger buffers during long training.

Increased target update interval. Raised to 2000 steps to reduce TD target oscillations and improve stability.

Sustained exploration. Exploration fraction kept at 0.20 with final epsilon 0.02 to maintain exploration in later stages.

Observed behaviour.

The agent reached a peak reward close to 500 and stabilised around 438 after 1M timesteps. It learned more structured behaviours, such as completing planets and chaining actions for higher rewards, while avoiding the collapse seen in higher learning rate configurations. However, generalisation remained limited to specific scenarios.

Reward shaping attempts.

Reward shaping was tested by adding penalties for death and time to discourage collisions and promote efficient behaviour.

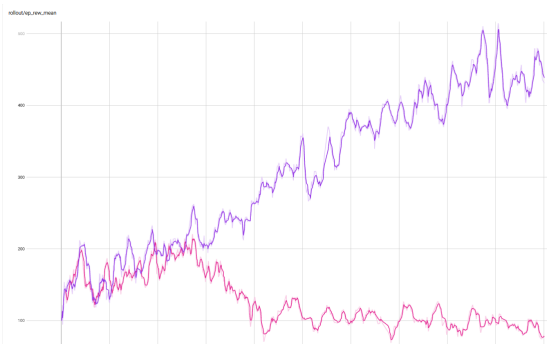

Figure 3: Comparison of learning curves (rollout/ep_rew_mean) between the best-performing configuration (purple) and the reward-shaped configuration with death penalties (pink). The baseline model reaches peak rewards close to 500 and stabilises around 438, while the penalised model stagnates around 100 reward.

This approach was counterproductive: it significantly reduced `rew_mean` and led to an overly conservative policy, where the agent avoided interaction and remained idle in safe regions.

Due to Gravitar's sparse rewards, penalising exploration discouraged the discovery of high-reward trajectories, leading to a suboptimal survival-only strategy. Consequently, reward shaping was discarded.

Interpretation.

These results show that in sparse-reward environments, **training stability is more important than aggressive learning**. While exp_04 performs best at 300k steps, the 1M-step experiment demonstrates that more conservative hyperparameters enable continued improvement. The optimal configuration therefore depends strongly on the training budget.

E. Future Directions

Prioritized Experience Replay (PER) [6]: PER samples transitions with probability proportional to their TD error, ensuring that rare but informative transitions are replayed more frequently. Given the importance of sample efficiency in sparse-reward environments, PER is a promising extension.

Dueling DQN [7]: The dueling architecture separates state-value and advantage estimation, improving learning in states where action choices have similar outcomes—common in Gravitar.

Extended training and stability: Results from the 1M-step experiment indicate that more conservative hyperparameters enable continued improvement beyond the 300k-step regime. Future work should further explore long-horizon training combined with improved sampling strategies.

.5. Conclusions

This paper presented a systematic evaluation of DQN on the ALE/Gravitar-v5 environment under a 300k-step training budget, comparing five configurations across three random seeds. Results show that the medium_balance configuration (exp_04) achieved the best performance (352.83 ± 33.20), outperforming the large-buffer setup. This indicates that balanced hyperparameter settings are more effective than scaling individual components such as replay buffer size. No single hyperparameter dominates performance in isolation. Large buffers improve stability but may reduce efficiency under limited budgets, while high learning rates introduce instability. Best results arise from a balanced combination of parameters. Extended experiments (1M timesteps) further showed that more conservative settings enable continued improvement, highlighting that optimal configurations depend on the training budget. Overall, hyperparameter interaction is critical, and iterative, data-driven tuning proves more effective than isolated adjustments.

References

[1] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, Feb. 2015.

[2] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Machine Learning*, vol. 8, pp. 293–321, 1992.

[3] A. Raffin *et al.*, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[4] M. Towers *et al.*, "Gymnasium," Farama Foundation, 2023. [Online]. Available: https://gymnasium.farama.org

[5] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013.

[6] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. ICLR*, 2016.

[7] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," in *Proc. ICML*, 2016.

[8] C. A. Sierra, "Machine Learning Challenge 1: Reinforcement Learning (DQN) for Atari," Universidad Distrital Francisco José de Caldas, Bogotá, 2026.